

ФГАОУ ВО «НИУ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Отчёт №2

Сортировка

(по дисциплине «Алгоритмы и структуры данных»)

Выполнил:

Лабин Макар Андреевич,
АиСД 3.2, Р3231, 466449.

Проверил:

Смирнов Виктор Игоревич

ИТМО

г. Санкт-Петербург, Россия
2026

Содержание

1	Яндекс.Контест	1
1.1	Коровы в стойла	1
1.1.1	Описание решения	1
1.1.2	Асимптотическая сложность	1
1.1.3	Листинг кода	1
1.2	Число	2
1.2.1	Описание решения	2
1.2.2	Асимптотическая сложность	2
1.2.3	Листинг кода	3
1.3	Кошмар в замке	3
1.3.1	Описание решения	3
1.3.2	Асимптотическая сложность	3
1.3.3	Листинг кода	4
1.4	Магазин	5
1.4.1	Описание решения	5
1.4.2	Асимптотическая сложность	5
1.4.3	Листинг кода	5
2	Timus	7
2.1	Median on the Plane	7
2.1.1	Описание решения	7
2.1.2	Асимптотическая сложность	7
2.1.3	Листинг кода	7
2.2	Country of Fools	9
2.2.1	Описание решения	9
2.2.2	Асимптотическая сложность	10
2.2.3	Листинг кода	10

1 Яндекс.Контест

1.1 Коровы в стойла

1.1.1 Описание решения

Применим стратегию бинарного поиска по ответу. Мы знаем, что ответ лежит в диапазоне от нуля до разности первого и последнего элементов исходного отсортированного массива. Предполагаем, что ответ лежит посередине данного отрезка. Если он подходит по условию, то двигаем левую границу к середине. Если нет — двигаем правую границу.

Проверить корректность предположений просто: достаточно пробежаться циклом по всему массиву и попытаться расставить коров хотя бы на предполагаемом расстоянии друг от друга. Если коров получилось не меньше, чем в условии — предположение верно, иначе — нет.

В ответ пойдёт максимальное из предположений искомого расстояния.

1.1.2 Асимптотическая сложность

Временная сложность решения — линейно-логарифмическая $\mathcal{O}(N \cdot \log N)$. Бинарный поиск происходит за логарифмическое число шагов, причём каждый шаг выполняется за линейное время из-за обхода всего массива.

Пространственная сложность решения — линейная $\mathcal{O}(N)$. Для поиска ответа входной массив данных сохраняется в памяти. Все промежуточные значения хранятся в константном количестве ячеек памяти.

1.1.3 Листинг кода

```
1 #include <iostream>
2 #include <vector>
3
4 int count_cows(const std::vector<long>& arr, int n, long dist) {
5     int count = 1;
6     long last = arr[0];
7     for (int i = 1; i < n; i++) {
8         if (std::abs(last - arr[i]) >= dist) {
9             last = arr[i];
10            count++;
11        }
12    }
13    return count;
14 }
15
16 long bin_search(const std::vector<long>& arr, int n, int k) {
17     if (n < 1) { // assertion
18         return -1;
19     }
20     long l = 0, r = arr[n - 1] - arr[0];
21     while (l + 1 < r) {
22         long mid = l + (r - l) / 2;
23         if (count_cows(arr, n, mid) < k) {
```

```

24     r = mid - 1;
25 } else {
26     l = mid;
27 }
28 }
29 if (count_cows(arr, n, r) >= k) {
30     return r;
31 }
32 return l;
33 }
34
35 int main() {
36     int N, K;
37     std::cin >> N >> K;
38     std::vector<long> arr;
39     arr.reserve(N);
40     for (int i = 0; i < N; i++) {
41         long item;
42         std::cin >> item;
43         arr.push_back(item);
44     }
45     std::cout << bin_search(arr, N, K);
46     return 0;
47 }

```

1.2 Число

1.2.1 Описание решения

Для решения задачи достаточно воспользоваться попарной лексикографической сортировкой строк в порядке убывания, а затем их конкатенировать. Такой подход гарантирует, что в старших разрядах окажутся цифры, имеющие больший лексикографический вес — а значит, и численный.

Стоит оговорить, что число всегда будет корректным с точки зрения записи. Это гарантирует условие: хотя бы в одной строке есть число с первой цифрой, отличной от нуля. Именно оно и будет первым за счёт большего лексикографического веса.

Для реализации попарной лексикографической сортировки строк воспользуемся стандартной сортировкой из библиотеки *STL* с изменённым компаратором, проверяющий лексикографический порядок конкретной пары строк.

1.2.2 Асимптотическая сложность

Временная сложность решения — линейно-логарифмическая $\mathcal{O}(N \cdot \log N)$. В алгоритме используется реализация сортировки из *STL*, которая имеет линейно-логарифмическую сложность согласно официальной документации.

Пространственная сложность решения — линейная $\mathcal{O}(N)$. Для сортировки массива требуется сначала его сохранить в памяти.

1.2.3 Листинг кода

```
1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4
5 int main() {
6     std::vector<std::string> arr;
7     std::string line;
8     while (std::cin >> line) {
9         arr.push_back(line);
10    }
11    struct {
12        bool operator()(std::string a, std::string b) const {
13            return a + b > b + a;
14        }
15    } customLess;
16    std::sort(arr.begin(), arr.end(), customLess);
17    for (std::string& s : arr) {
18        std::cout << s;
19    }
20    std::cout << "\n";
21    return 0;
22 }
```

1.3 Кошмар в замке

1.3.1 Описание решения

По условию задачи, на стоимость строки влияет каждая пара уникальных одинаковых символов. Если они встречаются чаще, чем два раза — третий и далее символы не вносят свой вклад, их можно размещать как угодно.

Значит, при вводе исходной строки стоит проанализировать её и подсчитать статистику по каждому символу: сколько раз он встречается в строке. Если он появился хотя бы два раза, его можно использовать для повышения стоимости строки.

Далее в задаче говорится, что стоимость считается как сумма произведений веса пары символов на максимальное расстояние между ними. Поступим жадно: пару символов, которая весит больше, разместим на большее расстояние. Чтобы этого достичь, будем размещать их по принципу палиндрома: i -ую по весу пару символов разместим на i -ое и $N - i$ -ое места. Оставшиеся символы добавим в центр в произвольном порядке — на стоимость строки это не повлияет.

1.3.2 Асимптотическая сложность

Временная сложность решения — линейная $\mathcal{O}(N)$. В алгоритме самая затратная по времени операция — посимвольная обработка строки: на каждый символ отводится константное число операций.

Полезная работа по максимизации стоимости строки выполняется за константное время, поскольку зависит от алфавита, который используется во входной строке — последний фиксирован по условию задачи.

Пространственная сложность решения — константная $O(1)$. Входная строка считывается посимвольно и не сохраняется в памяти целиком. Алфавит имеет фиксированный размер, поэтому на его хранение тоже требуется константное число ячеек.

1.3.3 Листинг кода

```
1 #include <algorithm>
2 #include <iostream>
3 #include <unordered_map>
4 #include <vector>
5
6 using letter = struct {
7     char c;
8     long v;
9 };
10
11 int main() {
12     std::unordered_map<char, long> occur;
13     char c;
14     while (std::cin >> std::noskipws >> c && c != '\n') {
15         occur[c] = occur.count(c) ? occur[c] + 1 : 1;
16     }
17     std::cin >> std::skipws;
18     std::vector<letter> costs;
19     costs.reserve(26);
20     for (int i = 0; i < 26; i++) {
21         letter l;
22         l.c = 'a' + i;
23         std::cin >> l.v;
24         costs.push_back(l);
25     }
26     std::sort(costs.begin(), costs.end(), [](const letter& a,
27         const letter& b) { return a.v > b.v; });
28     std::string pair, rest;
29     for (letter l : costs) {
30         if (occur.count(l.c) == 0) {
31             continue;
32         }
33         if (occur[l.c] > 1) {
34             pair += l.c;
35             for (int i = 0; i < occur[l.c] - 2; i++) {
36                 rest += l.c;
37             }
38             continue;
39         }
40         rest += l.c;
```

```

41     std::cout << pair << rest << std::string(pair.rbegin(), pair.
        rend());
42     return 0;
43 }

```

1.4 Магазин

1.4.1 Описание решения

Воспользуемся жадной стратегией: максимизируем стоимость самых дешёвых позиций в каждом чеке. Для определённости стоит отсортировать данный массив по возрастанию.

Если размер чека будет больше k , то массив получится разделить на меньшее число чеков без потери скидки. Ещё при таком подходе большой чек получит более дешёвые позиции, что уменьшает суммарную скидку. По этой причине не имеет смысла делить массив на части, кратные k .

В итоге остаётся делить массив на чеки размера k (кроме, возможно, последнего), начиная с правой части. В скидку пойдут крайние левые позиции из каждого чека размера k — она и будет максимальной.

1.4.2 Асимптотическая сложность

Временная сложность решения — линейно-логарифмическая $\mathcal{O}(N \cdot \log N)$. В алгоритме используется реализация сортировки из *STL*, которая имеет линейно-логарифмическую сложность согласно официальной документации.

Пространственная сложность решения — линейная $\mathcal{O}(N)$. Для сортировки массива требуется сначала его сохранить в памяти.

1.4.3 Листинг кода

```

1  #include <algorithm>
2  #include <iostream>
3  #include <vector>
4
5  int main() {
6      int n, k;
7      std::cin >> n >> k;
8      std::vector<int> arr;
9      arr.reserve(n);
10     long sum = 0;
11     for (int i = 0; i < n; i++) {
12         int item;
13         std::cin >> item;
14         arr.push_back(item);
15         sum += item;
16     }
17     std::sort(arr.begin(), arr.end());
18     for (int i = n - k; i >= 0; i -= k) {
19         sum -= arr[i];
20     }

```

```
21     std::cout << sum;
22     return 0;
23 }
```


2 Timus

2.1 Median on the Plane

2.1.1 Описание решения

Исходя из условия задачи, для любой точки $p_1 \in P$ можно выбрать $p_2 \in P$ так, что прямая p_1p_2 разделит точки из $P \setminus \{p_1, p_2\}$ на две равновеликие части. Это верно, потому что $|P| \geq 2$ и любые три точки неколлинеарны по условию.

Для определённости выберем p_1 с наименьшей абсциссой. Таких точек не более двух — выберем любую из них и исключим из исходного множества.

Далее отсортируем точки $p' \in P \setminus \{p_1\}$ по угловым коэффициентам прямых p_1p' . Здесь может возникнуть проблема: если в множестве $P \setminus \{p_1\}$ всё-таки есть точка $\tilde{p}_1$ с такой же абсциссой, как у p_1 , то угловой коэффициент прямой $p_1\tilde{p}_1$ стремится к $\pm\infty$. Аккуратно обработаем этот случай так, чтобы не возникло деления на ноль и при этом $\tilde{p}_1$ оказалась крайним элементом в отсортированном массиве.

Ответ к задаче — медиана отсортированного массива $P \setminus \{p_1\}$.

Замечание. Стоит отметить, что решение можно оптимизировать: достаточно найти $(\lfloor \frac{|P|}{2} \rfloor + 1)$ -ую порядковую статистику.

2.1.2 Асимптотическая сложность

Временная сложность решения — линейно-логарифмическая $\mathcal{O}(N \cdot \log N)$. Поиск точки p_1 с минимальной абсциссой линеен, операция удаления точки из массива тоже линейна. Остаётся сортировка массива, которая происходит за линейно-логарифмическое время.

Пространственная сложность решения — линейная $\mathcal{O}(N)$. В памяти хранится массив из всех введённых точек, который в дальнейшем обрабатывается с использованием константного числа переменных.

2.1.3 Листинг кода

```
1 #include <algorithm>
2 #include <cstdint>
3 #include <cstdint>
4 #include <iostream>
5 #include <iterator>
6 #include <stdexcept>
7 #include <vector>
8
9 class Fraction {
10 public:
11     Fraction(int num, int denum) : numerator_(num), denominator_(
12         denum) {
13
14     }
15
16     bool operator<(const Fraction& other) const {
17         return this->numerator_ * other.denominator_ < this->
18             denominator_ * other.numerator_;
```

```

16     }
17
18 private:
19     int64_t numerator_ = 1;
20     int64_t denominator_ = 1;
21 };
22
23 class Point {
24 public:
25     int x = 0;
26     int y = 0;
27     std::size_t idx = 0;
28
29     friend Fraction operator-(const Point& lpt, const Point& rpt)
30     {
31         return {rpt.y - lpt.y, rpt.x - lpt.x};
32     }
33 };
34 namespace {
35 Point FindLeftmostPoint(const std::vector<Point>& array) {
36     if (array.empty()) {
37         throw std::invalid_argument("array is empty");
38     }
39     Point left_point = array[0];
40     for (std::size_t i = 1; i < array.size(); i++) {
41         if (array[i].x < left_point.x) {
42             left_point = array[i];
43         }
44     }
45     return left_point;
46 }
47 } // namespace
48
49 int main() {
50     std::size_t n_points = 0;
51     std::cin >> n_points;
52
53     std::vector<Point> points;
54     points.reserve(n_points);
55     for (std::size_t i = 0; i < n_points; i++) {
56         Point curr_point;
57         std::cin >> curr_point.x >> curr_point.y;
58         curr_point.idx = i;
59         points.push_back(curr_point);
60     }
61
62     if (points.size() == 2) {
63         std::cout << 1 << " " << 2;
64         return 0;
65     }

```

```

66
67 Point left_point = {};
68 try {
69     left_point = FindLeftmostPoint(points);
70 } catch (const std::invalid_argument& e) {
71     std::cout << "Problem constraints are not met.";
72     return 1;
73 }
74 points.erase(std::next(points.begin(), static_cast<std::
    ptrdiff_t>(left_point.idx)));
75
76 std::sort(points.begin(), points.end(), [left_point](const
    Point& lpt, const Point& rpt) {
77     return (left_point - lpt) < (left_point - rpt);
78 });
79 std::cout << left_point.idx + 1 << " " << points[points.size()
    / 2].idx + 1;
80 return 0;
81 }

```

2.2 Country of Fools

2.2.1 Описание решения

По условию задачи необходимо вывести последовательность чисел, в которой будет больше всего пар различных элементов, идущих подряд. Если выводить числа циклично (*a-ля round-robin*), то в конце мы можем получить довольно длинную последовательность одинаковых элементов.

Исправим стратегию. Для начала отсортируем массив по убыванию количества знаков каждого типа. Заметим, что первые элементы массива при желании можно разделить на две категории:

- *знак-вышка (pillar)* — обладает уникальным значением количества, визуально выглядит как одиночная вышка;
- *знаки-равнины (plains)* — имеют общее для группы значение количества, визуально похожи на длинную (≥ 2) ступеньку.

В алгоритме *round-robin* на последнем этапе мы могли получить длинные цепочки элементов знака-вышки. Чтобы потенциально её уменьшить, нужно чередовать на каждой итерации вывод знака-вышки со знаками уровня ниже до тех пор, пока он не присоединится к знакам-равнинам.

Если на данной итерации знак-вышка слилась со знаками-равнинами, то на следующих итерациях достаточно вернуться к циклическому алгоритму: новых знаков-вышек не образуется. Отметим, что если на первой итерации первые элементы — знаки-равнины, то алгоритм *round-robin* отработает корректно.

Для реализации описанного алгоритма выполним предобработку отсортированного массива, чтобы сгруппировать знаки с одинаковым значением количества. Для удобства полученный артефакт будем хранить в двусвязном списке из двусвязных списков. Если какой-либо из списков пустеет, удаляем его из главного списка. Алгоритм завершится, когда главный список опустеет.

2.2.2 Асимптотическая сложность

Временная сложность решения — $\mathcal{O}(K \cdot \log K + N)$. Сортировка исходного массива из K элементов занимает линейно-логарифмическое время. Последующая обработка массива по заданному алгоритму — линейная, поскольку в сумме для вывода N элементов используется константное число операций, каждая из которых занимает константное время в случае двусвязного списка.

Пространственная сложность решения — $\mathcal{O}(K)$. В памяти хранятся два двусвязных списка из K элементов каждый. Числа выводятся в потоковом режиме, поэтому требуют константное количество памяти.

2.2.3 Листинг кода

```
1 #include <cstddef>
2 #include <forward_list>
3 #include <functional>
4 #include <iostream>
5 #include <iterator>
6 #include <list>
7 #include <stdexcept>
8
9 class Sign {
10 public:
11     std::size_t count = 0;
12     std::size_t type = 0;
13
14     // use this instead of `bool operator>(const Sign& other)
15     // const { ... }`
16     // otherwise i would have to define getters (POD vs Non-POD)
17     friend bool operator>(const Sign& lsn, const Sign& rsn) {
18         return lsn.count > rsn.count;
19     }
20 };
21
22 namespace {
23     std::list<std::list<Sign>> GenerateList(const std::forward_list<
24         Sign>& signs) {
25         if (signs.begin() == signs.end()) {
26             throw std::invalid_argument("forward list is empty");
27         }
28         std::list<std::list<Sign>> list = {{*signs.begin()}};
29         for (auto i = ++signs.begin(); i != signs.end() && i->count !=
30             0; i++) {
31             if (list.back().front().count == i->count) {
32                 list.back().push_front(*i);
33                 continue;
34             }
35             list.push_back(*i);
36         }
37         return list;
38     }
39 }
```

```

36
37 void StepPillar(std::list<std::list<Sign>>& list) {
38     // taking item from a pillar
39     auto next_list = std::next(list.begin());
40     auto top = list.begin()->begin();
41     std::cout << top->type << " ";
42     top->count--;
43     // if pillar is the only one item
44     if (next_list == list.end()) {
45         if (top->count == 0) {
46             list.pop_front();
47         }
48         return;
49     }
50     // if pillar has transformed into plain, extend next item
51     if (next_list->front().count == top->count) {
52         next_list->push_front(*top);
53         list.pop_front();
54     }
55     // take item from next plain
56     std::cout << next_list->back().type << " ";
57     next_list->back().count--;
58     const Sign hanging_sign = next_list->back();
59     next_list->pop_back();
60     // if hanging sign was the pillar
61     if (next_list->empty()) {
62         next_list = list.erase(next_list);
63     } else {
64         ++next_list;
65     }
66     if (hanging_sign.count == 0) {
67         return;
68     }
69     // extend next plain, if possible
70     if (next_list != list.end() && hanging_sign.count == next_list
71         ->front().count) {
72         next_list->push_front(hanging_sign);
73         // otherwise, create a new pillar
74     } else {
75         list.insert(next_list, {hanging_sign});
76     }
77 }
78
79 void ProcessPlain(std::list<std::list<Sign>>& list) {
80     auto next_list = std::next(list.begin());
81     auto plain = list.begin();
82     while (!plain->empty()) {
83         std::cout << plain->back().type << " ";
84         plain->back().count--;
85         // if item is not gone yet
86         if (plain->back().count > 0) {

```

```

86     // extend next plain, if possible
87     if (next_list != list.end() && plain->back().count ==
        next_list->front().count) {
88         next_list->push_front(plain->back());
89         // otherwise, create a new pillar
90     } else {
91         next_list = list.insert(next_list, {plain->back()});
92     }
93 }
94 plain->pop_back();
95 }
96 list.pop_front();
97 }
98 } // namespace
99
100 int main() {
101     std::size_t k_types = 1;
102     std::cin >> k_types;
103     std::forward_list<Sign> signs;
104     for (std::size_t i = 0; i < k_types; i++) {
105         Sign sign = {0, i + 1};
106         std::cin >> sign.count;
107         signs.push_front(sign);
108     }
109     signs.sort(std::greater<>());
110
111     std::list<std::list<Sign>> list = {};
112     try {
113         list = GenerateList(signs);
114     } catch (const std::invalid_argument& e) {
115         std::cout << "Problem constraints are not met.";
116         return 1;
117     }
118     signs.clear();
119
120     while (!list.empty() && std::next(list.front().begin()) ==
        list.front().end()) {
121         StepPillar(list);
122     }
123     while (!list.empty()) {
124         ProcessPlain(list);
125     }
126     return 0;
127 }

```